

Turn validated receipts into traceable rewards.

Build a small loyalty program with two roles and a clear approval rule.

MEMBER

Register and sign in

Submit receipts, track status, view vouchers

ADMINISTRATOR

Inspect purchase evidence

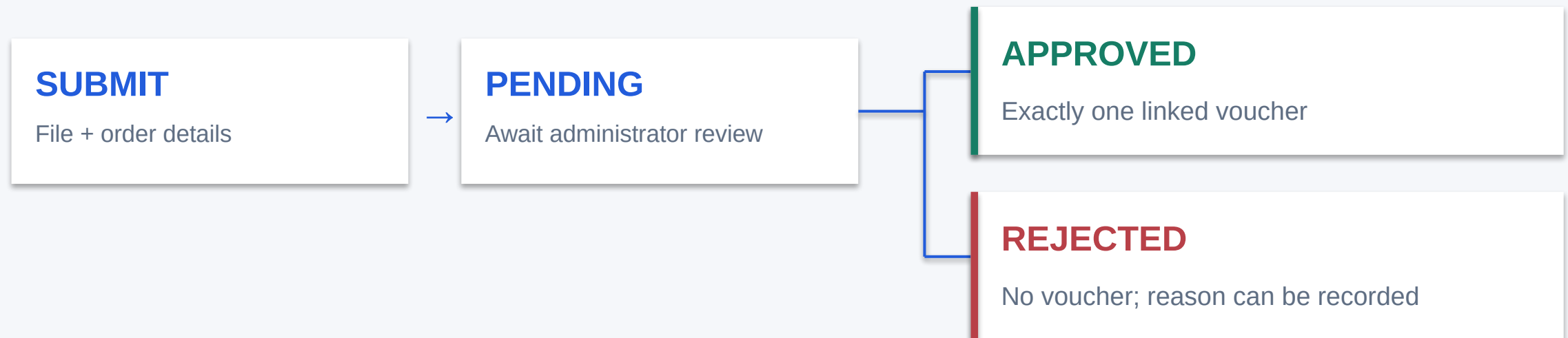
Approve or reject pending receipts

One approved receipt → one voucher. Rejection → no voucher.

Required stack: React · Node.js + Express · PostgreSQL

Every receipt starts pending.

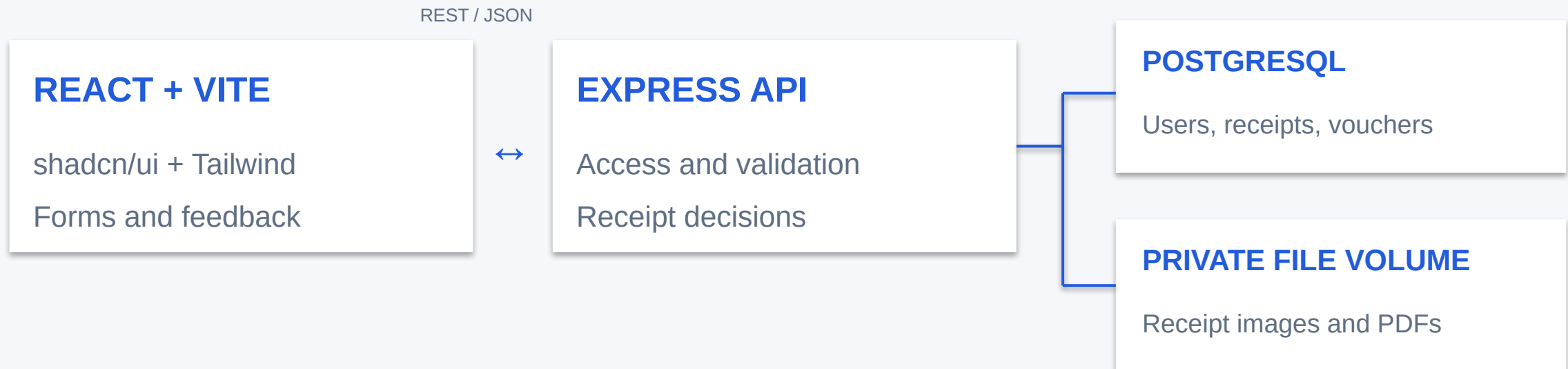
The administrator records one final decision; the member can follow the outcome.



A second decision returns 409 ALREADY_PROCESSED.

One application, with clear responsibilities.

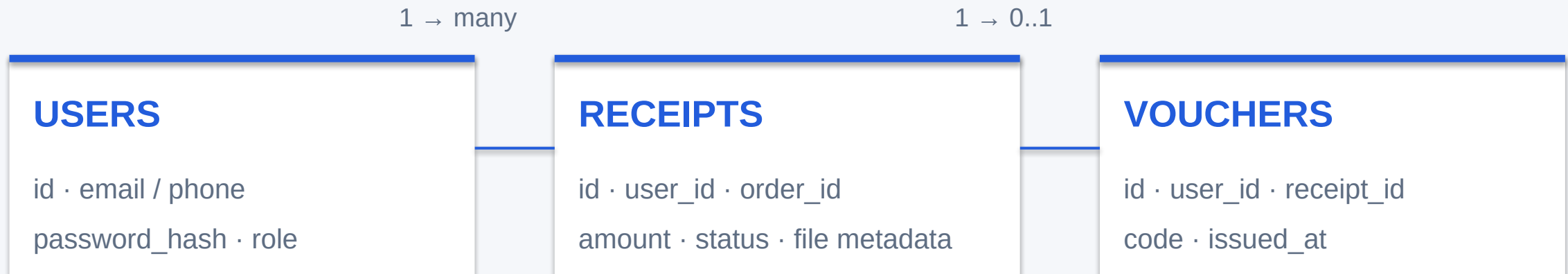
React presents the workflow. Express enforces the rules. PostgreSQL protects the records.



Raw SQL keeps the small data model and approval transaction easy to inspect.

Three tables keep each reward traceable.

Identity → purchase evidence → issued reward



Unique order ID per member

`UNIQUE(user_id, order_id)`

At most one voucher per receipt

`UNIQUE(vouchers.receipt_id)`

Approval and voucher creation commit together.

The critical path is a database transaction, protected by a row lock.

01**BEGIN**

Start transaction

02**LOCK**

SELECT ... FOR
UPDATE

03**CHECK**

Receipt must be
PENDING

04**WRITE**

Approve + insert
voucher

05**COMMIT**

Persist both changes

Any failure → ROLLBACK | Repeated decision → 409 | Rejection → no insert

Check identity, permission, ownership and input.

The browser helps the user; the API decides whether an action is allowed.

IDENTITY	bcrypt hashing (cost 12) · JWT lifetime: eight hours
PERMISSION	Current account role loaded from PostgreSQL on each request
OWNERSHIP	Own receipts and vouchers only; owner/admin file downloads
INPUT & FILES	Positive amount, valid date, required fields · allowed types ≤ 5 MB

401 Sign in required 403 Access denied 400 Invalid input 409 Conflict

Test the happy path and the failure cases.

Recorded verification: 2 September 2026 · Local Docker deployment

9 / 9

AUTOMATED TESTS

Validation and approval service

53

API REQUESTS

Full Postman acceptance flow

137 / 137

ASSERTIONS PASSED

Ownership, decisions and errors

Also checked: Docker startup, schema/seed, production build and HTTPS deployment.

Easy to run. Clear to review and extend.

A small deployment with explicit tradeoffs and documented AI assistance.

REPRODUCIBLE DELIVERY

make → Docker → schema + admin seed

CI tests/builds → image → VPS health check

MY CONTRIBUTION

Technical brief and requirements

UX direction, review, UAT and final decisions

NEXT ITERATION

Object storage · versioned migrations · rate limiting · session revocation

Let's follow one receipt from submission to reward.

I'll switch between a member session and an administrator session.

- | | | |
|----|--------------------------|--|
| 01 | Member | Upload a receipt and show PENDING in history. |
| 02 | Administrator | Open the evidence, then approve the receipt. |
| 03 | Member | Refresh history and show the linked voucher. |
| 04 | Alternate outcome | Reject a second receipt; confirm no extra voucher. |

antlysis-loyalty.axelyn.com

01

Member submits a receipt

Upload the file and purchase details. The receipt enters the admin review queue.

Outcome: PENDING

02

Admin approves a receipt

Review the evidence and approve. The member receives exactly one linked voucher.

Outcome: APPROVED + ONE VOUCHER

03

Admin rejects a receipt

Review the evidence and reject. The member sees the decision and any recorded reason.

Outcome: REJECTED + NO VOUCHER